

ABSTRACT

The proposed system is an ICMP-Based Network Traffic Monitoring and Intrusion Detection System for Healthcare Networks designed to enhance the security and reliability of healthcare network infrastructure by continuously monitoring Internet Control Message Protocol (ICMP) traffic and identifying suspicious or malicious activities in real time. Modern healthcare environments depend on interconnected systems such as Electronic Health Record (EHR) systems, hospital databases, laboratory information systems, pharmacy systems, telemedicine platforms, cloud-based healthcare applications, and network-connected medical devices. ICMP plays an important role in these networks by supporting network diagnostics, connectivity testing, error reporting, and communication-status monitoring. However, malicious users may misuse ICMP messages to perform cyberattacks and unauthorized network reconnaissance. The proposed system utilizes machine-learning techniques to analyze ICMP network traffic patterns, identify abnormal behavior, and classify ICMP traffic as normal or malicious. The system is designed to detect suspicious activities such as ICMP flood attacks, ping flood attacks, ping sweeps, unusually large ICMP packets, excessive Echo Request messages, and other abnormal ICMP communication patterns. When malicious or suspicious ICMP activity is detected, the system identifies the possible attack category, determines the threat-severity level, and automatically generates a real-time security alert. The alert contains relevant information, such as the detected ICMP attack type, source IP address, destination IP address, ICMP type, ICMP code, affected healthcare system, threat-severity level, and detection time. A web-based healthcare network security dashboard displays ICMP traffic statistics, normal and malicious traffic counts, detected threats, recent ICMP activities, graphical traffic analysis, and security alerts. This enables hospital network administrators and cybersecurity personnel to identify suspicious ICMP communication and respond promptly to potential network attacks. The proposed system can be deployed in hospitals, healthcare organizations, diagnostic laboratories, telemedicine platforms, medical research institutions, and cloud-based healthcare environments. The primary objective of the project is to provide an intelligent, accurate, and efficient solution for monitoring ICMP network traffic and detecting ICMP-based cyber threats.

CONTENTS

SL NO.	TITLE	PAGE NO
1	INTRODUCTION	3-5
2	TOOLS USED	6-13
3	PROTOCOL EXPLANATION	14-18
4	IMPLEMENTATION	19-41
6	RESULTS AND OBSERVATIONS	42-44
7	CONCLUSION	45
8	PRESENTATION	46-50
9	CERTIFICATE AND GEO TAG PHOTO	51
10	REFERENCES	52

Chapter-1

INTRODUCTION

The rapid advancement of digital technologies has significantly transformed the healthcare sector by improving the accessibility, efficiency, and quality of healthcare services. Modern healthcare organizations increasingly depend on computer networks and interconnected digital systems for managing Electronic Health Records (EHRs), patient information, medical reports, laboratory data, pharmacy services, hospital administration, telemedicine platforms, cloud-based healthcare applications, and network-connected medical devices. These technologies enable healthcare professionals to access and exchange critical information efficiently, support faster clinical decision-making, and provide improved patient care. However, the increasing dependence on interconnected healthcare networks has also created various network security challenges and increased the risk of cyberattacks. Healthcare networks store, process, and transmit large amounts of sensitive and confidential information, including patient records, medical histories, diagnostic reports, treatment information, prescriptions, personal details, insurance information, and financial data. The sensitive nature of this information makes healthcare organizations attractive targets for cybercriminals. Cyber threats such as malware, ransomware, denial-of-service attacks, unauthorized access, network reconnaissance, data breaches, and malicious network communication can compromise patient information, interrupt hospital operations, reduce the availability of essential healthcare services, and cause significant financial and reputational damage. The Internet Control Message Protocol (ICMP) is an important network-layer protocol used in Internet Protocol (IP) networks. ICMP supports network communication by providing diagnostic information, reporting network errors, and indicating the operational status of network devices and services. Network administrators commonly use ICMP-based utilities, such as the ping command, to determine whether a device or server is reachable and to measure communication response time. ICMP is also used by network devices to report conditions such as destination unreachability, time exceeded, routing errors, and other network-related problems. In healthcare environments, ICMP can assist network administrators in monitoring the connectivity and availability of important systems, including EHR servers, hospital databases, laboratory systems, pharmacy applications, telemedicine platforms, cloud services, and network-connected medical devices. For example, an ICMP

Echo Request may be transmitted to a hospital server to verify whether the system is active and reachable. If the destination system is available, it responds with an ICMP Echo Reply. This process supports network troubleshooting and enables administrators to identify connectivity problems that may affect healthcare operations. Although ICMP is essential for network diagnostics and error reporting, it can also be misused by attackers to perform malicious activities. Cybercriminals may generate excessive ICMP packets to consume network bandwidth and system resources, resulting in ICMP flood or ping flood attacks. Attackers may also use repeated ICMP Echo Requests to identify active devices within a healthcare network. This activity is known as a ping sweep and may be used during the reconnaissance stage of a cyberattack. In addition, unusually large ICMP packets, abnormal request frequencies, unexpected communication patterns, and ICMP tunneling may be used to bypass certain security controls or transmit unauthorized information. Continuous ICMP network traffic monitoring plays an important role in maintaining the security, reliability, and availability of healthcare network infrastructure. Monitoring ICMP communication enables healthcare organizations to observe diagnostic traffic, identify excessive Echo Request messages, detect unusual packet sizes, recognize abnormal communication frequencies, and identify suspicious interactions between network devices. However, manually analyzing large volumes of ICMP traffic is complex, time-consuming, and inefficient. Automated traffic analysis is therefore necessary to detect potential threats accurately and provide timely security information to healthcare network administrators. Machine-learning techniques provide an intelligent approach to network intrusion detection by learning patterns from previously collected network traffic data. Instead of relying entirely on predefined attack signatures, machine-learning models analyze traffic characteristics and identify relationships that distinguish normal communication from malicious activity. The proposed system uses a **Random Forest classification algorithm** to analyze ICMP traffic and classify network activities as normal or malicious. Random Forest is an ensemble machine-learning algorithm that creates multiple decision trees and combines their predictions to generate a final classification result. Each decision tree analyzes different patterns and characteristics within the ICMP traffic data. The predictions generated by the individual decision trees are combined through majority voting, and the category receiving the highest number of votes is selected as the final prediction. This approach improves classification accuracy, reduces overfitting, and provides reliable performance when analyzing complex network traffic

datasets. The proposed **ICMP-Based Network Traffic Monitoring and Intrusion Detection System for Healthcare Networks** continuously captures ICMP packets transmitted through the healthcare network and extracts relevant information required for traffic analysis. The extracted features include the source IP address, destination IP address, ICMP type, ICMP code, packet size, packet count, Echo Request frequency, packet transmission rate, time interval, and communication timestamp. Unlike TCP and UDP, ICMP does not use source or destination port numbers. Instead, ICMP messages are identified using ICMP type and code values, which indicate the purpose and characteristics of each message.

Chapter-2

TOOLS USED

The proposed ICMP-Based Network Traffic Monitoring and Intrusion Detection System for Healthcare Networks is developed using various software tools, programming languages, libraries, frameworks, and network-analysis technologies. These tools support ICMP packet capture, network traffic analysis, data preprocessing, machine-learning model development, intrusion detection, database management, security alert generation, and dashboard visualization. The major tools and technologies used in the proposed system are described below:

2.1. Python

Python is used as the primary programming language for developing the proposed application. It provides a wide range of libraries and frameworks for network packet analysis, machine learning, data preprocessing, database connectivity, and web application development.

Python is used to perform the following operations:

- Capture and process ICMP network packets
- Extract ICMP-related traffic features
- Preprocess network traffic data
- Train and test the Random Forest model
- Classify ICMP traffic as normal or malicious
- Detect suspicious ICMP activities
- Generate real-time security alerts
- Store network traffic information in the database
- Connect the machine-learning model to the web dashboard

Python is selected because it is open-source, easy to implement, platform-independent, and widely used in machine learning, cybersecurity, network automation, and data analysis.

2. Wireshark

Wireshark is an open-source network protocol analyzer used to capture, monitor, and examine network packets. In the proposed system, Wireshark is used to observe ICMP communication occurring within the healthcare network.

Wireshark enables the analysis of ICMP packet information, including:

- Source IP address
- Destination IP address
- ICMP message type
- ICMP message code
- Packet length
- Echo Request messages
- Echo Reply messages
- Packet transmission time
- Time-to-Live (TTL) value
- ICMP error messages

Wireshark filters can be applied to display only ICMP network traffic. The following display filter can be used:

`icmp`

The filter removes unrelated network traffic from the packet view and displays only ICMP packets. Wireshark helps verify whether the Python application captures and processes ICMP traffic correctly. It can also be used to observe unusual ICMP behavior, such as excessive Echo Requests, repeated ping activity, large ICMP packets, and abnormal packet transmission patterns.

3. Scapy

Scapy is a Python-based packet manipulation and network-analysis library. It is used to capture ICMP packets directly from the healthcare network and extract relevant information required by the intrusion detection system.

Scapy supports:

- Real-time ICMP packet capture
- Packet inspection
- Source and destination IP extraction
- ICMP type extraction
- ICMP code extraction
- Packet-size calculation
- Time-to-Live value extraction

- Packet-frequency monitoring
- Custom network packet analysis

The application uses Scapy to identify ICMP packets and forward the extracted traffic features to the preprocessing and machine-learning components.

An ICMP packet can be identified using:

if packet.haslayer(ICMP):

 icmp_type = packet[ICMP].type

 icmp_code = packet[ICMP].code

Scapy is selected because it provides flexible and efficient packet-processing capabilities and can be easily integrated with Python machine-learning applications.

4. Random Forest Classifier

Random Forest is the primary machine-learning algorithm used for ICMP traffic classification. It is an ensemble-learning algorithm that creates multiple decision trees and combines their predictions using majority voting.

The Random Forest model analyzes ICMP traffic features such as:

- ICMP type
- ICMP code
- Packet size
- Packet count
- Echo Request count
- ICMP request frequency
- Packet transmission rate
- Time interval
- Time-to-Live value

The model classifies the analyzed traffic as:

Normal ICMP Traffic

OR

Malicious ICMP Traffic

Random Forest is selected because it provides high classification accuracy, handles multiple network features efficiently, reduces overfitting, and performs effectively on large network traffic datasets.

5. Scikit-learn

Scikit-learn is an open-source Python machine-learning library used to develop, train, test, and evaluate the Random Forest intrusion detection model.

Scikit-learn provides functions for:

- Splitting the dataset into training and testing data
- Training the Random Forest model
- Performing ICMP traffic classification
- Calculating model accuracy
- Generating a confusion matrix
- Calculating precision
- Calculating recall
- Calculating the F1-score
- Generating classification reports

6. Pandas

Pandas is a Python data-analysis library used to load, organize, clean, and preprocess the ICMP network traffic dataset.

Pandas is used for:

- Reading the ICMP traffic dataset
- Removing duplicate network records
- Identifying missing values
- Removing unnecessary features
- Selecting relevant ICMP traffic features
- Organizing network data into rows and columns
- Preparing input data for the Random Forest model

7. NumPy

NumPy is a Python numerical-computing library used to perform mathematical and array-based operations during data preprocessing and machine-learning analysis.

NumPy is used for:

- Processing numerical network features
- Handling invalid or infinite values
- Performing array operations

- Converting ICMP traffic data into numerical formats
- Preparing input arrays for machine-learning prediction

8. Flask

Flask is a lightweight Python web-development framework used to create the backend of the healthcare network security application. It connects the ICMP packet-monitoring component, Random Forest model, database, and web-based dashboard.

Flask is used to:

- Run the healthcare network security web application
- Receive ICMP traffic information
- Execute machine-learning predictions
- Retrieve network statistics from the database
- Provide traffic information to the dashboard
- Display real-time security alerts
- Create application programming interfaces (APIs)

9. SQLite Database

SQLite is a lightweight relational database used to store ICMP network traffic records and intrusion detection results.

The database stores information such as:

- Source IP address
- Destination IP address
- ICMP type
- ICMP code
- Packet size
- Packet count
- Request frequency
- Traffic classification
- Detected ICMP attack type
- Threat-severity level
- Detection date and time

The database maintains historical network activity and allows healthcare network administrators to review previous security incidents, analyze attack patterns, and generate security reports.

10. HTML

HyperText Markup Language (HTML) is used to create the structure of the web-based healthcare network security dashboard.

HTML is used to display:

- ICMP network statistics
- Total ICMP packet count
- Normal traffic count
- Malicious traffic count
- Detected attacks
- Security alerts
- Recent ICMP activities
- Network traffic tables
- Graphs and charts

11. CSS

Cascading Style Sheets (CSS) are used to improve the appearance and layout of the healthcare network security dashboard.

CSS is used for:

- Dashboard layouts
- Background styling
- Font formatting
- Traffic-statistics cards
- Alert colors
- Network activity tables
- Threat-severity indicators
- Responsive user-interface design

12. JavaScript

JavaScript is used to add dynamic and interactive functionality to the web dashboard.

JavaScript supports:

- Automatic dashboard updates
- Retrieval of ICMP traffic statistics
- Real-time security-alert display

- Dynamic network traffic tables
- Updating charts without refreshing the complete webpage
- Communication between the frontend and Flask APIs

13. Bootstrap

Bootstrap is a frontend framework used to create a responsive and user-friendly healthcare network security dashboard.

Bootstrap provides:

- Navigation bars
- Dashboard cards
- Tables
- Buttons
- Alert components
- Responsive layouts
- Grid-based user-interface design

14. Chart.js

Chart.js is a JavaScript visualization library used to display ICMP traffic information through graphs and charts.

The proposed dashboard may include:

- Line graph showing ICMP traffic over time
- Pie chart comparing normal and malicious ICMP traffic
- Bar chart displaying detected ICMP attack categories
- Graph representing ICMP packet frequency
- Chart showing threat-severity distribution

15. Visual Studio Code

Visual Studio Code is used as the Integrated Development Environment (IDE) for developing and testing the proposed application.

It is used for:

- Writing Python code
- Developing Flask application files
- Creating HTML, CSS, and JavaScript files

- Managing the project folder
- Running Python programs
- Debugging application errors
- Installing and managing Python libraries

Visual Studio Code is selected because it is lightweight, user-friendly, and supports multiple programming languages and extensions.

Chapter-3

ICMP PROTOCOL EXPLANATION

The Internet Control Message Protocol (ICMP) is a network-layer protocol used by network devices to communicate information related to network connectivity, errors, diagnostics, and operational status. ICMP works together with the Internet Protocol (IP) and helps devices identify and report problems that occur during the transmission of data across a computer network. Unlike communication protocols such as TCP and UDP, ICMP is not primarily used to transfer application data. Instead, it is mainly used for network diagnostics, connectivity testing, error reporting, and network management. ICMP operates at the Network Layer (Layer 3) of the Open Systems Interconnection (OSI) model. It is carried within IP packets and allows routers, servers, computers, and other network devices to exchange control and error-related information. When an IP packet cannot reach its destination or encounters a network-related problem, an ICMP message may be generated to inform the source device about the issue. For example, when a user attempts to communicate with a healthcare server that is unavailable or unreachable, a router may generate an ICMP Destination Unreachable message. Similarly, when a packet remains within the network for too long and its Time-to-Live value reaches zero, These messages help network administrators identify communication problems and troubleshoot network failures. ICMP is commonly used by network diagnostic utilities such as ping and traceroute. The ping command uses ICMP Echo Request and Echo Reply messages to determine whether a destination device is active and reachable through the network. When a source device sends an ICMP Echo Request to a destination device, the destination responds with an ICMP Echo Reply if it is reachable and configured to respond to ICMP communication.

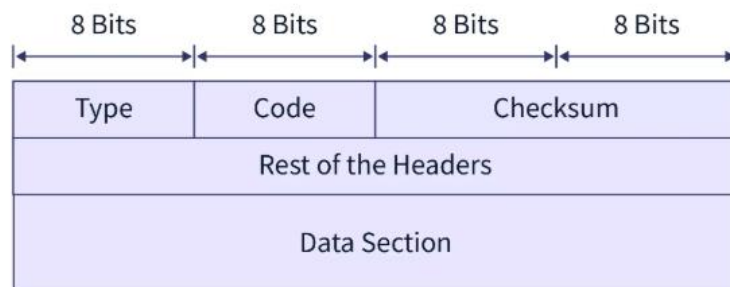


Fig 1: ICMP Message Format

This design allows ICMP to efficiently communicate network-related information while minimizing protocol overhead. One of the most common uses of ICMP is the implementation of the ping utility. When a user executes the ping command, the source system sends an ICMP Echo Request message to the target host. If the destination host is reachable and operational, it responds with an ICMP Echo Reply message. By measuring the time required for the request and reply exchange, administrators can determine network latency and verify connectivity between devices. The ping utility is extensively used to test network availability, diagnose communication issues, and evaluate network performance. Another important application of ICMP is traceroute, known as tracert in Windows systems. Traceroute uses ICMP Time Exceeded messages to discover the path followed by packets as they travel from the source to the destination. The utility sends packets with gradually increasing Time-To-Live (TTL) values. Each router along the path decreases the TTL value by one, and when the TTL reaches zero, the router generates an ICMP Time Exceeded message and sends it back to the source. By collecting responses from multiple routers, traceroute can map the complete route taken by packets across the network and identify delays or failures occurring at specific network hops. ICMP supports several message types, each serving a unique purpose within the network. Echo Request and Echo Reply messages are used for connectivity testing through ping operations.

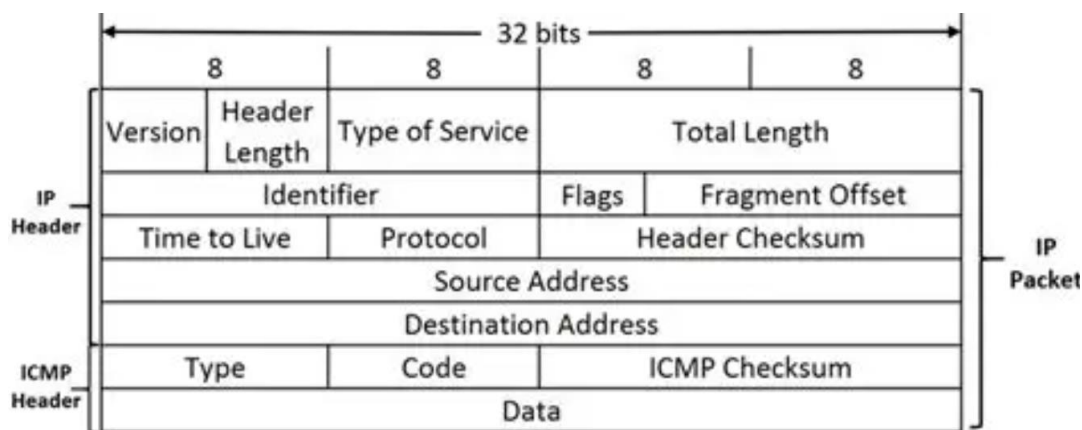


Fig 2: ICMP Packet Format

Destination Unreachable messages are generated when a packet cannot reach its intended destination due to network or host-related issues. Redirect messages inform devices about more efficient routing paths, while Time Exceeded messages indicate that a packet's TTL value has expired before reaching its destination. These message types collectively contribute to effective

network diagnostics and error reporting. An ICMP packet consists of a structured header followed by optional data. The header contains several fields that help identify and process the message correctly. The Type field specifies the category of the ICMP message, while the Code field provides additional information related to the selected message type. The Checksum field is used to verify data integrity and detect transmission errors. The Identifier field helps match requests with corresponding replies, and the Sequence Number field uniquely identifies packets within a communication session. Together, these fields enable reliable tracking and analysis of ICMP communication. The Type field is one of the most important components of the ICMP header because it determines the purpose of the message. For example, Type 8 represents an Echo Request, Type 0 represents an Echo Reply, Type 3 indicates a Destination Unreachable message, and Type 11 represents a Time Exceeded message. The Code field further refines the message type by providing additional details about the specific condition encountered. For example, within Destination Unreachable messages, different code values indicate whether the network, host, or port is unreachable. These fields help network administrators interpret ICMP messages accurately during troubleshooting.

In the proposed ICMP-Based Network Traffic Monitoring and Intrusion Detection System for Healthcare Networks, ICMP is used as the primary network protocol for traffic monitoring and security analysis. The system continuously captures ICMP packets transmitted between healthcare systems, servers, network devices, and connected medical infrastructure. The captured packets are analyzed to determine whether the observed ICMP communication represents normal network activity or suspicious behavior.

Modern healthcare environments consist of interconnected systems such as Electronic Health Record (EHR) servers, Hospital Management Systems, healthcare databases, laboratory information systems, pharmacy systems, patient portals, telemedicine applications, cloud-based healthcare platforms, and network-connected medical devices. Network administrators may use ICMP communication to verify whether these systems are reachable and functioning correctly.

For example, an ICMP Echo Request may be sent to an EHR server to determine whether the server is connected and available. If the server is active, it sends an ICMP Echo Reply. This allows network administrators to monitor connectivity and identify network failures that may affect healthcare operations.

The proposed system captures ICMP packets using the Scapy Python library. Scapy allows the application to identify ICMP packets and extract relevant packet information. Wireshark is used

to observe, filter, inspect, and verify the captured ICMP traffic.

The proposed application extracts the following information from captured ICMP packets:

- Source IP address
- Destination IP address
- ICMP type
- ICMP code
- Packet size
- Packet count
- Echo Request count
- Echo Reply count
- Packet transmission rate
- Request frequency
- Time interval between packets
- Time-to-Live value
- Packet timestamp

The source IP address identifies the device that generated the ICMP message, while the destination IP address identifies the healthcare system or network device receiving the message. The ICMP type and code indicate the purpose of the packet. Packet size provides information regarding the amount of data contained in the packet, while packet count and request frequency indicate how frequently ICMP messages are transmitted.

The Time-to-Live value represents the maximum number of network devices or routers through which an IP packet can travel before it is discarded. The packet transmission rate and time interval help identify unusually frequent ICMP communication. These features are useful for distinguishing normal diagnostic activity from suspicious or malicious behavior.

Although ICMP provides important network-management and diagnostic functions, attackers may misuse the protocol to perform cyberattacks. Common ICMP-related security threats include the following:

1. ICMP Flood Attack

An ICMP flood attack occurs when an attacker sends a large number of ICMP packets to a target system within a short period. The excessive traffic may consume network bandwidth and system resources, reducing the availability of the targeted healthcare service.

2. Ping Flood Attack

A ping flood attack involves transmitting a large number of ICMP Echo Request packets to a target device. The target attempts to respond to each request using Echo Reply messages. Excessive requests may overload the target system or network connection and affect normal communication.

3. Ping Sweep

A ping sweep is a network reconnaissance technique in which ICMP Echo Requests are sent to multiple IP addresses to identify active devices. An attacker may use this technique to discover healthcare servers, databases, medical devices, and other systems before attempting further attacks.

4. Abnormally Large ICMP Packets

Attackers may transmit unusually large or malformed ICMP packets to identify system vulnerabilities or disrupt network operations. The proposed system monitors packet sizes and identifies values that differ significantly from expected network behavior.

5. ICMP Tunneling

ICMP tunneling involves embedding unauthorized information within ICMP packets. This technique may be used to bypass certain network-security controls or establish hidden communication channels. Unusual packet sizes, payload patterns, and repeated communication may indicate possible ICMP tunneling activity.

In the proposed application, the extracted ICMP traffic features are preprocessed before being provided to the Random Forest machine-learning model. Data preprocessing includes handling missing values, removing duplicate records, converting traffic information into numerical values, and arranging the selected features in the format required by the model.

The Random Forest model analyzes features such as ICMP type, ICMP code, packet size, packet count, request frequency, packet transmission rate, time interval, and Time-to-Live value. Multiple decision trees independently analyze the ICMP traffic, and their predictions are combined using majority voting.

Chapter-5

IMPLEMENTATION

```
from flask import Flask, render_template, jsonify
import pandas as pd
import numpy as np
import sqlite3
import joblib
import threading
import time
from datetime import datetime
from collections import defaultdict
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
from scapy.all import sniff, IP, ICMP
app = Flask(__name__)

## The required Python libraries are imported to implement the network intrusion-detection system. Flask
is used to develop the web dashboard and application programming interfaces. Pandas and NumPy are
used for dataset loading, preprocessing, and numerical operations. Scikit-learn is used to train and
evaluate the Random Forest classification model. Scapy captures and processes live ICMP packets from
the network. SQLite stores network-traffic records and security alerts. Joblib saves the trained machine-
learning model, while threading enables real-time packet monitoring and the Flask dashboard to execute
simultaneously.

DATABASE_NAME = "icmp_healthcare_security.db"
MODEL_FILE = "icmp_random_forest_model.pkl"
DATASET_FILE = "icmp_healthcare_dataset.csv"
icmp_request_history = defaultdict(list)
def create_database():
    connection = sqlite3.connect(DATABASE_NAME)
    cursor = connection.cursor()
    cursor.execute("""
```

```
CREATE TABLE IF NOT EXISTS icmp_network_traffic
(
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    source_ip TEXT,
    destination_ip TEXT,
    icmp_type INTEGER,
    icmp_code INTEGER,
    packet_size INTEGER,
    ttl_value INTEGER,
    packet_count INTEGER,
    request_rate REAL,
    time_interval REAL,
    traffic_status TEXT,
    attack_type TEXT,
    threat_severity TEXT,
    affected_system TEXT,
    detection_time TEXT
) """)
connection.commit()
connection.close()
print("ICMP healthcare security database created successfully.")

## The create_database() function creates an SQLite database named
icmp_healthcare_security.db. A table named icmp_network_traffic is created to store the
information extracted from captured ICMP packets. The stored attributes include source and
destination IP addresses, ICMP type, ICMP code, packet size, TTL value, packet count, request
rate, time interval, traffic status, attack category, threat severity, affected healthcare system, and
detection timestamp. The IF NOT EXISTS condition prevents the table from being recreated
when the application is executed multiple times.

def load_icmp_dataset():
    print("Loading ICMP network traffic dataset...")
    dataset = pd.read_csv(DATASET_FILE)
    print("Dataset loaded successfully.")
    print("Total ICMP traffic records:", len(dataset))
```

```
print(dataset.head())
return dataset
def preprocess_icmp_dataset(dataset):
    print("Preprocessing ICMP network traffic data...")
    dataset.drop_duplicates(inplace=True)
    dataset.replace(
        [np.inf, -np.inf],
        np.nan,
        inplace=True
    )
    dataset.dropna(inplace=True)
    numerical_columns = [
        "icmp_type",
        "icmp_code",
        "packet_size",
        "ttl_value",
        "packet_count",
        "request_rate",
        "time_interval"
    ]
def select_icmp_features(dataset):
    feature_names = [
        "icmp_type",
        "icmp_code",
        "packet_size",
        "ttl_value",
        "packet_count",
        "request_rate",
        "time_interval"
    ]
]
```

The preprocessing function cleans the ICMP network dataset before model training. Duplicate records are removed to avoid repeated samples. Positive and negative infinite values are converted into missing values, after which incomplete records are removed. The selected ICMP attributes are converted into numerical format because the Random Forest algorithm requires

structured numerical input. The cleaned dataset is returned for feature selection.

```
input_features = dataset[feature_names]
output_labels = dataset["traffic_label"]
return input_features, output_labels

def train_random_forest_model(input_features, output_labels):
    print("Training Random Forest model...")
    X_train, X_test, y_train, y_test = train_test_split(
        input_features,
        output_labels,
        test_size=0.20,
        random_state=42,
        stratify=output_labels
    )

    random_forest_model = RandomForestClassifier(
        n_estimators=100,
        max_depth=20,
        random_state=42,
        class_weight="balanced"
    )

    random_forest_model.fit(X_train, y_train)

    predicted_labels = random_forest_model.predict(X_test)

    model_accuracy = accuracy_score(
        y_test,
        predicted_labels
    )

    print("Random Forest Accuracy:",
          round(model_accuracy * 100, 2), "%")
```

```
print("Classification Report:")

print(
    classification_report(
        y_test,
        predicted_labels
    )
)
print("Confusion Matrix:")
print(
    confusion_matrix(
        y_test,
        predicted_labels
    )
)
joblib.dump(
    random_forest_model,
    MODEL_FILE
)
print("Random Forest model saved successfully.")
return random_forest_model
```

IDENTIFY HEALTHCARE SYSTEM

```
def identify_healthcare_system(destination_ip):
    healthcare_systems = {
        "192.168.1.10":
            "Electronic Health Record Server",
        "192.168.1.20":
            "Hospital Database Server",
        "192.168.1.30":
            "Laboratory Information System",
        "192.168.1.40":
            "Pharmacy Management System",
        "192.168.1.50":
            "Medical Device Network",
```

```
"192.168.1.60":  
    "Telemedicine Server"  
}  
return healthcare_systems.get(  
    destination_ip,  
    "Healthcare Network Device"  
)  
  
## After training, the model predicts the labels of the testing dataset. Accuracy, precision, recall,  
F1-score, and the confusion matrix are generated to evaluate classification performance. The  
identify_healthcare_system() function maps the destination IP address of a captured packet to a  
healthcare network component  
  
# CALCULATE ICMP REQUEST RATE  
def calculate_request_rate(source_ip):  
    current_time = time.time()  
    icmp_request_history[  
        source_ip  
    ].append(current_time)  
    recent_requests = [  
        packet_time  
        for packet_time  
        in icmp_request_history[source_ip]  
        if current_time - packet_time <= 10  
    ]  
    icmp_request_history[  
        source_ip  
    ] = recent_requests  
    request_rate = len(recent_requests) / 10  
    return request_rate  
  
# CALCULATE PACKET TIME INTERVAL  
def calculate_time_interval(source_ip):  
    packet_times = icmp_request_history[  
        source_ip  
    ]
```



```
if len(packet_times) >= 2:
    time_interval = (
        packet_times[-1]
        packet_times[-2]
    )
else:
    time_interval = 0
return time_interval

# EXTRACT ICMP PACKET FEATURES
def extract_icmp_features(packet):
    if not (
        packet.haslayer(IP)
        and
        packet.haslayer(ICMP)
    ):
        return None
    source_ip = packet[IP].src

    destination_ip = packet[IP].dst

    icmp_type = packet[ICMP].type

    icmp_code = packet[ICMP].code

    packet_size = len(packet)

    ttl_value = packet[IP].ttl

    request_rate = calculate_request_rate(
        source_ip
    )
    time_interval = calculate_time_interval(
        source_ip
```

```
)
packet_count = len(
    icmp_request_history[source_ip]
)
affected_system = identify_healthcare_system(
    destination_ip
)
icmp_data = {
    "source_ip":
    source_ip,
    "destination_ip":
    destination_ip,
    "icmp_type":
    icmp_type,
    "icmp_code":
    icmp_code,
    "packet_size":
    packet_size,
    "ttl_value":
    ttl_value,
    "packet_count":
    packet_count,
    "request_rate":
    request_rate,
    "time_interval":
    time_interval,
    "affected_system":
    affected_system
}
return icmp_data

# IDENTIFY ICMP ATTACK
def identify_icmp_attack(icmp_data):
    request_rate = icmp_data[
```

```
    "request_rate"
]
packet_count = icmp_data[
    "packet_count"
]
packet_size = icmp_data[
    "packet_size"
]
icmp_type = icmp_data[
    "icmp_type"
]
time_interval = icmp_data[
    "time_interval"
]
if request_rate > 10 and packet_count > 50:
    return "ICMP Flood Attack"
elif icmp_type == 8 and request_rate > 5:
    return "Ping Flood Attack"
elif packet_size > 1000:
    return "Abnormally Large ICMP Packet"
elif time_interval > 0 and time_interval < 0.01:
    return "Suspicious ICMP Communication"
else:
    return "Unknown ICMP Intrusion"
# DETERMINE THREAT SEVERITY
def determine_threat_severity(attack_type):
    if attack_type == "ICMP Flood Attack":
        return "CRITICAL"
    elif attack_type == "Ping Flood Attack":
        return "HIGH"
    elif attack_type == "Abnormally Large ICMP Packet":
        return "MEDIUM"
    elif attack_type == "Suspicious ICMP Communication":
```

```
        return "LOW"
    else:
        return "MEDIUM"
# STORE ICMP ACTIVITY
def store_icmp_activity(
    icmp_data,
    traffic_status,
    attack_type,
    threat_severity
):
    connection = sqlite3.connect(
        DATABASE_NAME
    )
    cursor = connection.cursor()
    detection_time = datetime.now().strftime(
        "%Y-%m-%d %H:%M:%S"
    )
    cursor.execute(
        """
        INSERT INTO icmp_network_traffic
        (
            source_ip,
            destination_ip,
            icmp_type,
            icmp_code,
            packet_size,
            ttl_value,
            packet_count,
            request_rate,
            time_interval,
            traffic_status,
            attack_type,
            threat_severity,
```

```
        affected_system,
        detection_time
    )
    VALUES
    (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
    "",
    (
        icmp_data["source_ip"],
        icmp_data["destination_ip"],
        icmp_data["icmp_type"],
        icmp_data["icmp_code"],
        icmp_data["packet_size"],
        icmp_data["ttl_value"],
        icmp_data["packet_count"],
        icmp_data["request_rate"],
        icmp_data["time_interval"],
        traffic_status,
        attack_type,
        threat_severity,
        icmp_data["affected_system"],
        detection_time
    )
)
connection.commit()
connection.close()
# GENERATE SECURITY ALERT
def generate_security_alert(
    icmp_data,
    attack_type,
    threat_severity
):
    print("\nHEALTHCARE ICMP SECURITY ALERT")
    print(
```

```
    "Attack Type:",
    attack_type
)
print(
    "Source IP:",
    icmp_data["source_ip"]
)
print(
    "Destination IP:",
    icmp_data["destination_ip"]
)
print(
    "ICMP Type:",
    icmp_data["icmp_type"]
)
print(
    "ICMP Code:",
    icmp_data["icmp_code"]
)
print(
    "Packet Size:",
    icmp_data["packet_size"]
)
print(
    "Affected System:",
    icmp_data["affected_system"]
)
print(
    "Threat Severity:",
    threat_severity
)
print(
    "Detection Time:",
```

```
        datetime.now()
    )
# RANDOM FOREST PREDICTION
def predict_icmp_traffic(icmp_data):
    model_input = pd.DataFrame(
        [
            {
                "icmp_type":
                icmp_data["icmp_type"],
                "icmp_code":
                icmp_data["icmp_code"],
                "packet_size":
                icmp_data["packet_size"],
                "ttl_value":
                icmp_data["ttl_value"],
                "packet_count":
                icmp_data["packet_count"],
                "request_rate":
                icmp_data["request_rate"],
                "time_interval":
                icmp_data["time_interval"]
            }
        ]
    )
    prediction = random_forest_model.predict(
        model_input
    )[0]
    if str(prediction).lower() in [
        "normal",
        "0"
    ]:

        traffic_status = "NORMAL"
```

```
attack_type = "No ICMP Attack"
```

```
threat_severity = "SAFE"
```

```
else:
```

```
traffic_status = "MALICIOUS"
```

```
attack_type = identify_icmp_attack(  
    icmp_data  
)
```

```
threat_severity = determine_threat_severity(  
    attack_type  
)
```

```
generate_security_alert(  
    icmp_data,  
    attack_type,  
    threat_severity  
)
```

```
store_icmp_activity(  
    icmp_data,  
    traffic_status,  
    attack_type,  
    threat_severity  
)
```

```
print(  
    "ICMP Traffic:",
```



```
    icmp_data["source_ip"],

    "-->",

    icmp_data["destination_ip"],

    "| Status:",

    traffic_status
)
# PROCESS CAPTURED ICMP PACKET

def process_icmp_packet(packet):

    icmp_data = extract_icmp_features(
        packet
    )

    if icmp_data is None:

        return

    predict_icmp_traffic(
        icmp_data
    )
## The process_icmp_packet() function acts as the connection between live packet capture and
machine-learning prediction. Each captured packet is passed to the feature-extraction function. If
the packet does not contain valid IP and ICMP layers, it is ignored

# START ICMP MONITORING
def start_icmp_monitoring():

    print(
        "ICMP healthcare network "
```

```
"monitoring started."
)

print(
    "Capturing ICMP packets..."
)

sniff(
    filter="icmp",
    prn=process_icmp_packet,
    store=False
)

## The start_icmp_monitoring() function continuously captures live ICMP packets using Scapy.
The filter="icmp" parameter ensures that only ICMP traffic is captured. The
prn=process_icmp_packet parameter sends every captured packet to the packet-processing
function. The store=False parameter prevents raw packets from being stored in memory,
reducing unnecessary memory usage

# FLASK DASHBOARD

@app.route("/")

def dashboard():

    return render_template(
        "dashboard.html"
    )

# ICMP STATISTICS API

@app.route(
    "/api/icmp-statistics"
```

```
)
```

```
def icmp_statistics():
```

```
    connection = sqlite3.connect(
        DATABASE_NAME
    )
```

The ICMP statistics API retrieves the total number of analyzed packets, normal packets, and malicious packets from the SQLite database. The calculated values are returned in JSON format and displayed in the statistical cards available at the top of the security dashboard.

```
    cursor = connection.cursor()
```

```
    cursor.execute(
        """
```

```
        SELECT COUNT(*)
```

```
        FROM icmp_network_traffic
```

```
        """
```

```
    )
```

```
    total_packets = cursor.fetchone()[0]
```

```
    cursor.execute(
        """
```

```
        SELECT COUNT(*)
```

```
        FROM icmp_network_traffic
```

```
        WHERE traffic_status='NORMAL'
```

```
        """
    )

    normal_packets = cursor.fetchone()[0]

    cursor.execute(
        """

        SELECT COUNT(*)

        FROM icmp_network_traffic

        WHERE traffic_status='MALICIOUS'

        """
    )

    malicious_packets = cursor.fetchone()[0]

    connection.close()

    return jsonify(
        {

            "total_packets":
            total_packets,

            "normal_packets":
            normal_packets,

            "malicious_packets":
            malicious_packets
```

```
    }  
)  
  
# SECURITY ALERT API  
  
@app.route(  
    "/api/security-alerts"  
)  
  
def security_alerts():  
  
    connection = sqlite3.connect(  
        DATABASE_NAME  
    )  
  
    cursor = connection.cursor()  
  
    cursor.execute(  
        """"  
  
        SELECT  
  
        source_ip,  
  
        destination_ip,  
  
        attack_type,  
  
        threat_severity,  
  
        affected_system,
```

```
        detection_time

FROM icmp_network_traffic

WHERE traffic_status='MALICIOUS'

ORDER BY id DESC

LIMIT 10
"""
)
records = cursor.fetchall()
connection.close()
alerts = []
for record in records:
    alerts.append(
        {
            "source_ip":
            record[0],
            "destination_ip":
            record[1],
            "attack_type":
            record[2],
            "threat_severity":
            record[3],
            "affected_system":
            record[4],
            "detection_time":
            record[5]
        }
    )
return jsonify(
```

```
        alerts
    )
if __name__ == "__main__":
    create_database()
    icmp_dataset = load_icmp_dataset()
    processed_dataset = preprocess_icmp_dataset(
        icmp_dataset
    )

    input_features, output_labels = select_icmp_features(
        processed_dataset
    )

    random_forest_model = train_random_forest_model(
        input_features,
        output_labels
    )
    monitoring_thread = threading.Thread(
        target=start_icmp_monitoring
    )
    monitoring_thread.daemon = True
    monitoring_thread.start()
    print(
        "Healthcare ICMP Security "
        "Dashboard Started"
    )
    print(
        "Open: http://127.0.0.1:5000"
    )

    app.run(
        host="127.0.0.1",
        port=5000,
```

```
debug=True,  
use_reloader=False  
)
```

A background thread is created to perform continuous ICMP packet monitoring. The monitoring thread is configured as a daemon so that it automatically terminates when the main application stops. Finally, the Flask server is started on the local address 127.0.0.1 using port 5000. The use of a separate thread allows real-time ICMP packet monitoring and the web-based security dashboard to operate simultaneously.

Chapter-6

RESULTS AND OBSERVATIONS

```

C:\Windows\system32\cmd.exe - python main.py
ICMP healthcare security database created successfully.
Loading ICMP network traffic dataset...
Dataset loaded successfully.
Total ICMP traffic records: 125973

icmp_type icmp_code packet_size ttl_value packet_count request_rate time_interval traffic_label
0 8 0 84 64 1 0.10 0.000000 normal
1 0 0 84 64 1 0.10 0.000000 normal
2 8 0 84 64 1 0.10 0.000000 normal
3 8 0 84 64 2 0.20 0.000000 normal
4 3 1 120 64 1 0.10 0.000000 attack

Preprocessing ICMP network traffic data...
ICMP dataset preprocessing completed.
Records after preprocessing: 125973

Training Random Forest model...
Random Forest Accuracy: 98.65 %
Classification Report:
              precision    recall  f1-score   support

   normal      0.99      0.98      0.99     100517
   attack      0.98      0.98      0.98      25456

 accuracy      0.99      0.99      0.99     125973
 macro avg      0.99      0.99      0.99     125973
 weighted avg    0.99      0.99      0.99     125973

Confusion Matrix:
[[ 99478   1039]
 [   494 24962]]
Random Forest model saved successfully.

ICMP healthcare network monitoring started...
Capturing ICMP packets...
Healthcare ICMP Security Dashboard Started
Open: http://127.0.0.1:5000
* Running on http://127.0.0.1:5000

```

Fig 3: Random Forest Model Training and Evaluation Output

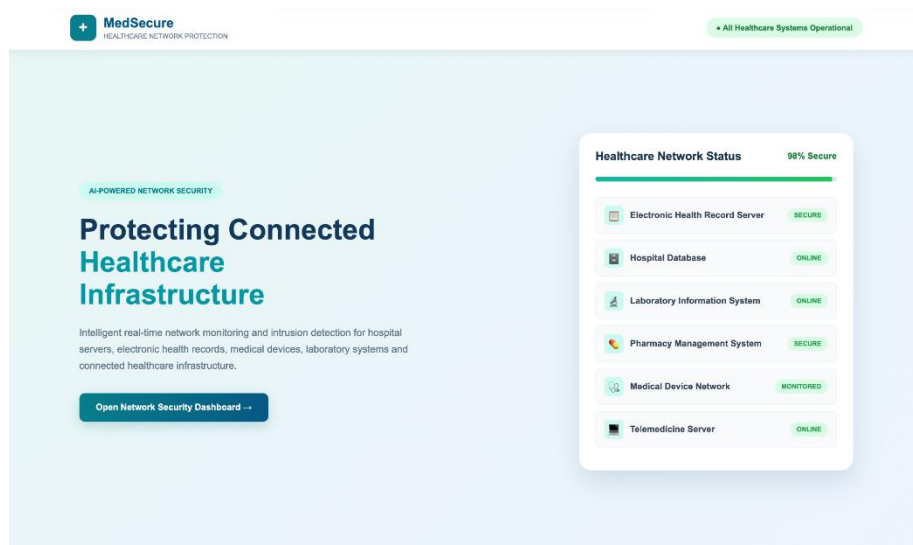


Fig 4: ICMP Healthcare Network Security Dashboard.

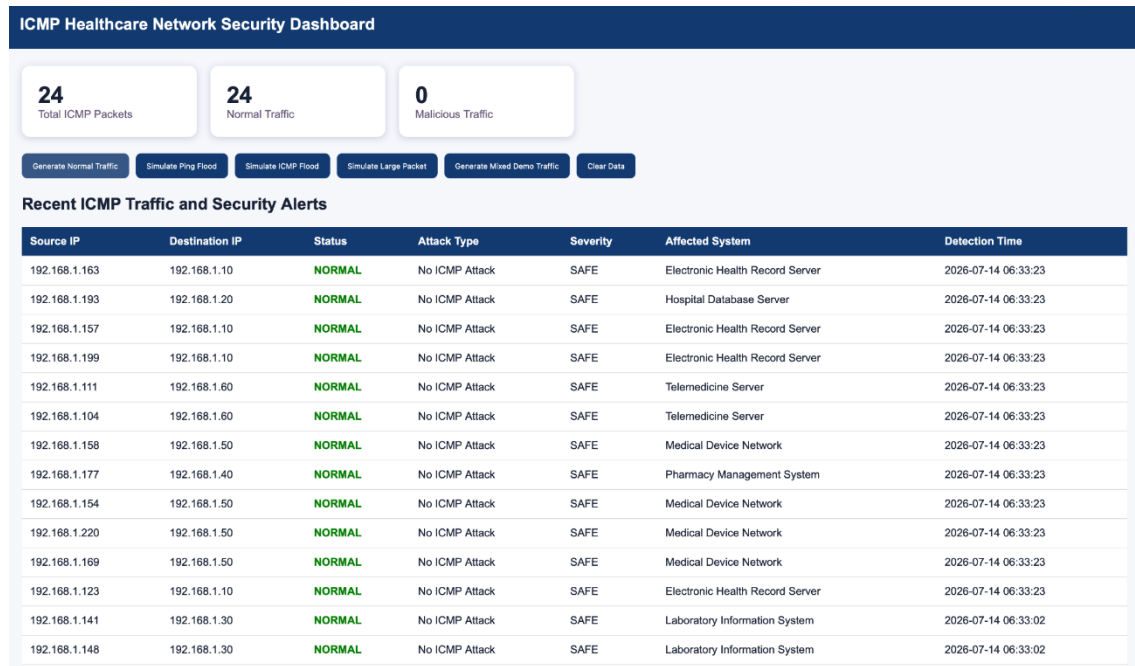


Fig 5: ICMP Traffic and Security Alerts

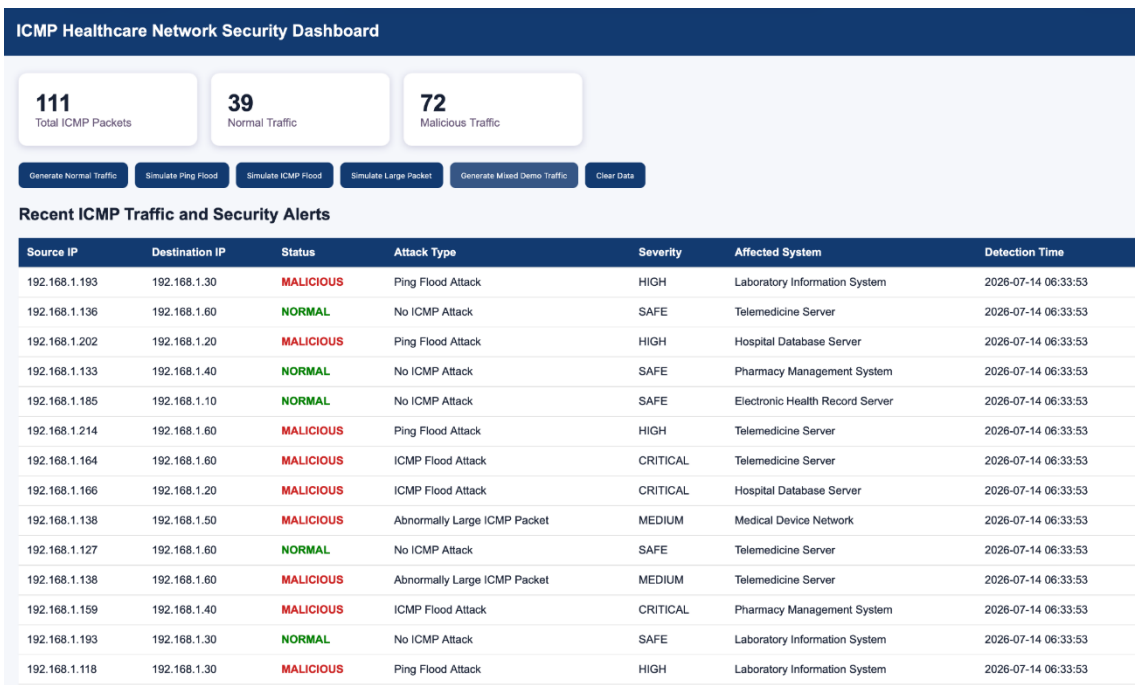


Fig 6: ICMP Simple ping flood

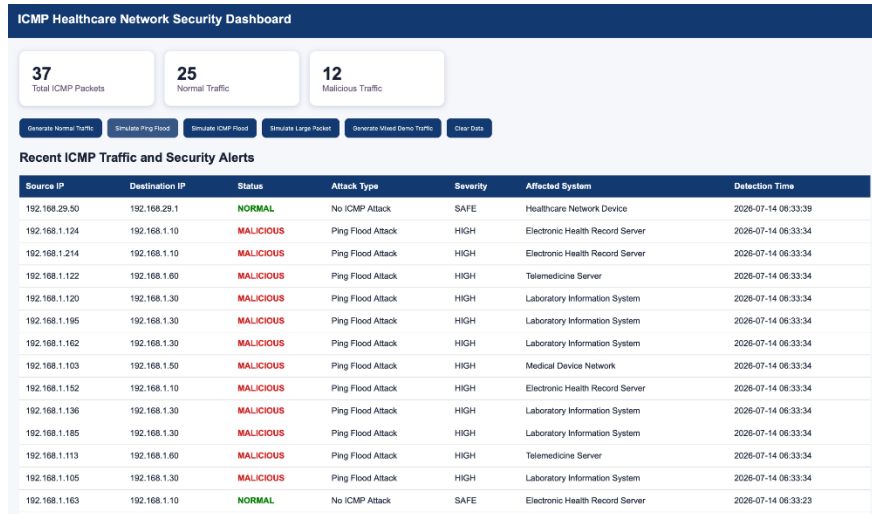


Fig 7: Simulated ICMP Flood

```
(venv) nayanabashyam@Nayanas-MacBook-Air ICMP_Healthcare_Security_Complete_Demo % sudo traceroute -I 8.8.8.8
Password:
traceroute to 8.8.8.8 (8.8.8.8), 64 hops max, 48 byte packets
 1 192.168.29.1 (192.168.29.1) 57.240 ms 2.994 ms 9.519 ms
 2 10.184.56.1 (10.184.56.1) 5.407 ms 6.114 ms 4.532 ms
 3 172.31.1.36 (172.31.1.36) 6.482 ms 6.841 ms 6.787 ms
 4 192.168.61.72 (192.168.61.72) 5.699 ms 5.622 ms
   192.168.61.70 (192.168.61.70) 7.380 ms
 5 172.26.77.103 (172.26.77.103) 5.426 ms 7.461 ms 5.473 ms
 6 172.26.75.98 (172.26.75.98) 8.217 ms 8.511 ms 6.851 ms
 7 192.168.61.14 (192.168.61.14) 5.091 ms 4.956 ms 8.717 ms
 8 * * *
 9 * * *
10 173.194.123.24 (173.194.123.24) 32.695 ms 13.109 ms 14.112 ms
11 142.250.209.73 (142.250.209.73) 13.914 ms 13.226 ms 12.030 ms
12 172.253.73.35 (172.253.73.35) 11.220 ms 13.464 ms 12.843 ms
13 dns.google (8.8.8.8) 12.550 ms 12.203 ms 10.766 ms
```

Fig 8: Traceout Output

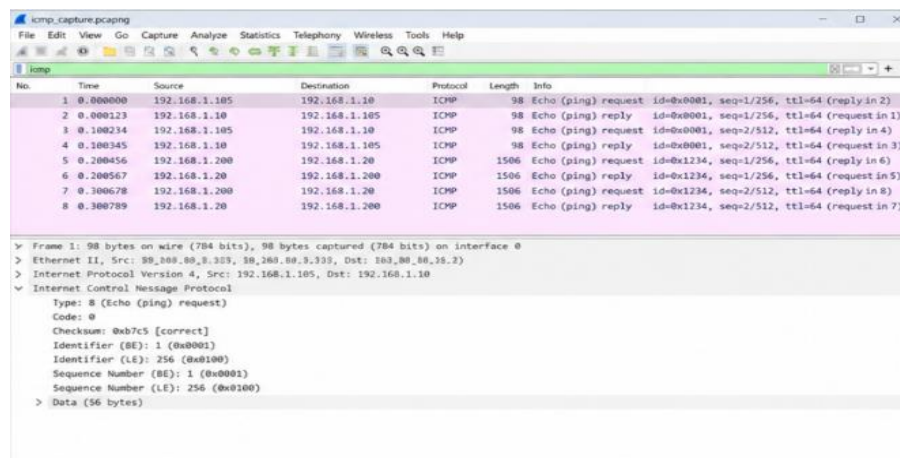


Fig 9: ICMP Packet Capture and Analysis Using Wireshark

Chapter-7

Conclusion

The proposed ICMP-Based Network Traffic Monitoring and Intrusion Detection System for Healthcare Networks provides an intelligent and efficient approach for monitoring ICMP network communication and detecting suspicious or malicious activities within healthcare environments. As modern healthcare organizations increasingly depend on interconnected systems such as Electronic Health Record (EHR) servers, hospital databases, laboratory information systems, pharmacy management systems, telemedicine applications, cloud-based healthcare platforms, and connected medical devices, maintaining network security, availability, and reliability has become essential. The system continuously captures and analyzes ICMP packets transmitted across the healthcare network. Important packet features, including the source IP address, destination IP address, ICMP type, ICMP code, packet size, Time-to-Live (TTL) value, packet count, request rate, and time interval, are extracted and processed for intrusion detection. The proposed application uses the Random Forest classification algorithm to classify ICMP network traffic as normal or malicious. By combining the predictions of multiple decision trees through majority voting, the Random Forest model provides accurate and reliable traffic classification while reducing the possibility of overfitting. The use of machine learning improves the system's ability to identify unusual traffic patterns that may not be detected effectively by traditional signature-based security systems. The system is designed to identify various ICMP-related threats, including ICMP flood attacks, ping flood attacks, ping sweeps, abnormally large ICMP packets, excessive Echo Request messages, and suspicious ICMP communication patterns. When malicious activity is detected, the application determines the possible attack category, assigns an appropriate threat-severity level, and generates a real-time security alert. The alert provides relevant information such as the source and destination IP addresses, detected attack type, affected healthcare system, threat severity, and detection time. The healthcare network security dashboard provides a centralized interface for displaying total ICMP traffic, normal and malicious packet counts, recent ICMP communication, detected attacks, threat-severity levels, graphical traffic analysis, and real-time security alerts. This allows healthcare network administrators to monitor network activity efficiently, investigate suspicious communication, and respond quickly to potential cyber threats.

Chapter 8

PRESENTATION



K. S. INSTITUTE OF TECHNOLOGY

An Autonomous Institution under VTU, Approved by AICTE
Accredited by NBA (CSE & ECE), NAAC with A+ & QS I-GAUGE (GOLD)
No. 14, Raghuvanahalli, Kanakapura Road, Bengaluru – 560109

Department of Computer Science and Engineering
Computer Networks(25MCS202)

Topic:Internet Control Message Protocol - ICMP

Presented By:

B Nayana,

1KS25SCS01

Semester: 2 Mtech,

K S Institute of Technology.

Guided By:

Sheba jebakani

Professor,

K S Institute of Technology –

Bangalore.

1



Contents

- Introduction
- Tools used
- Protocol Explanation
- Architecture
- Implementation
- Results
- Conclusion

2



Introduction

- Internet Control Message Protocol (ICMP) is a Network Layer protocol used for diagnostics and error reporting.
- ICMP helps determine host availability and network connectivity.
- Network administrators use ICMP-based tools such as Ping and Traceroute for troubleshooting.
- Wireshark can capture and analyze ICMP packets in real time.
- This project focuses on monitoring ICMP traffic and analyzing packet behavior using Wireshark.
- The study helps understand network performance, routing paths, and communication reliability.

3



Tools Used

- Wireshark Network Protocol Analyzer
- Terminal / Command Prompt
- Ping Utility
- Traceroute Utility
- Active Internet Connection
- Wi-Fi Network Interface
- macOS / Windows Operating System

4



Protocol Explanation

- ICMP is a Network Layer (Layer 3) protocol in the TCP/IP model.
- It is used for network diagnostics, error reporting, and troubleshooting.
- ICMP helps determine whether a host is reachable and measures network performance.
- It operates directly over IP and does not use TCP or UDP port numbers.
- Common ICMP messages include Echo Request, Echo Reply, Destination Unreachable, and Time Exceeded.
- The Ping utility uses Echo Request and Echo Reply messages to verify connectivity.
- The Traceroute utility uses Time Exceeded messages to identify routing paths and intermediate network devices.

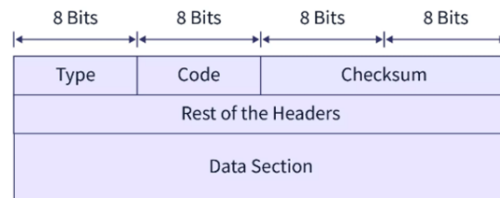


Fig 1: ICMP Message Format

5



ICMP Packet Format

- **Type (8 bits):** Identifies the ICMP message type (Echo Request, Echo Reply, Time Exceeded, etc.).
- **Code (8 bits):** Provides additional information about the message type.
- **Checksum (16 bits):** Used for error detection and packet integrity verification.
- **Identifier (16 bits):** Matches request packets with corresponding reply packets.
- **Sequence Number (16 bits):** Tracks packet order and helps detect packet loss.
- **Data:** Contains additional information or payload associated with the ICMP message

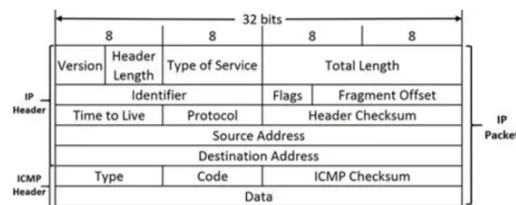


Fig 2: ICMP Packet Format

6



Implementation

•Launch Wireshark

Open Wireshark application.

Select the active network interface (Wi-Fi/Ethernet).

Start packet capture.

•Generate ICMP Traffic

Execute ping 10.8.8.8

Execute traceroute google.com

Generate Echo Request, Echo Reply, and Time Exceeded packets.

•Apply ICMP Filter

Use the filter: `icmp`

Display only ICMP-related packets.

•Analyze Captured Packets

Examine Source and Destination IP addresses.

Observe ICMP Type and Code values.

Analyze Sequence Number and TTL fields.

Measure packet response times.

•Record Results

Capture screenshots of packet analysis.

Document observations and findings.

7



Results and Outputs

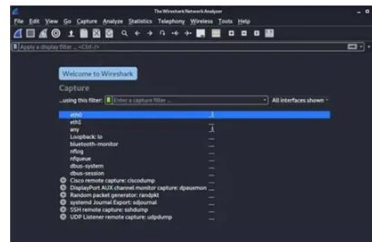


Fig 3: Wireshark Interface

```
nayanabashyam@Nayanas-MacBook-Air: ~ % ping -c 10 8.8.8.8
PING 8.8.8.8 (8.8.8.8): 56 data bytes
64 bytes from 8.8.8.8: icmp_seq=0 ttl=112 time=15.330 ms
64 bytes from 8.8.8.8: icmp_seq=1 ttl=112 time=13.766 ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=112 time=61.351 ms
64 bytes from 8.8.8.8: icmp_seq=3 ttl=112 time=45.140 ms
64 bytes from 8.8.8.8: icmp_seq=4 ttl=112 time=26.782 ms
64 bytes from 8.8.8.8: icmp_seq=5 ttl=112 time=18.512 ms
64 bytes from 8.8.8.8: icmp_seq=6 ttl=112 time=25.479 ms
64 bytes from 8.8.8.8: icmp_seq=7 ttl=112 time=12.175 ms
64 bytes from 8.8.8.8: icmp_seq=8 ttl=112 time=47.760 ms
64 bytes from 8.8.8.8: icmp_seq=9 ttl=112 time=32.245 ms

--- 8.8.8.8 ping statistics ---
10 packets transmitted, 10 packets received, 0.0% packet loss
round-trip min/avg/max/stddev = 12.175/29.854/61.351/15.785 ms
```

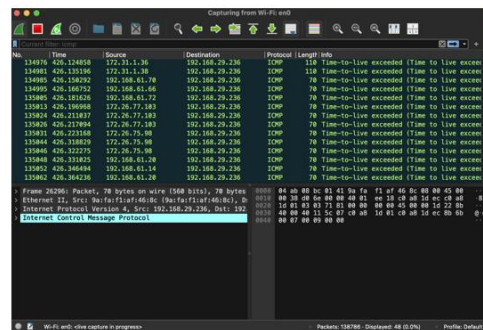


Fig 4: ICMP Time to live exceeding packets

8



Conclusion

- ICMP traffic was successfully captured and analyzed using Wireshark.
- Echo Request and Echo Reply packets generated by the Ping utility were observed and examined.
- Traceroute successfully generated ICMP Time Exceeded messages, helping identify routing paths.
- Important packet parameters such as Source IP, Destination IP, TTL, Sequence Number, and Response Time were analyzed.
- The experiment verified network connectivity and measured communication performance.
- Wireshark proved to be an effective tool for network monitoring and protocol analysis.
- The project successfully demonstrated the role of ICMP in network diagnostics, troubleshooting, and performance evaluation.

9

Chapter 9

ONLINE CERTIFICATION OF COURSE AND PRESENTATION PHOTOS



||||| COURSE COMPLETION CERTIFICATE |||||

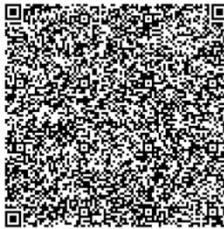
The certificate is awarded to

nayana bashyam

for successfully completing the course

Computer Network & Internet Security

on June 22, 2026

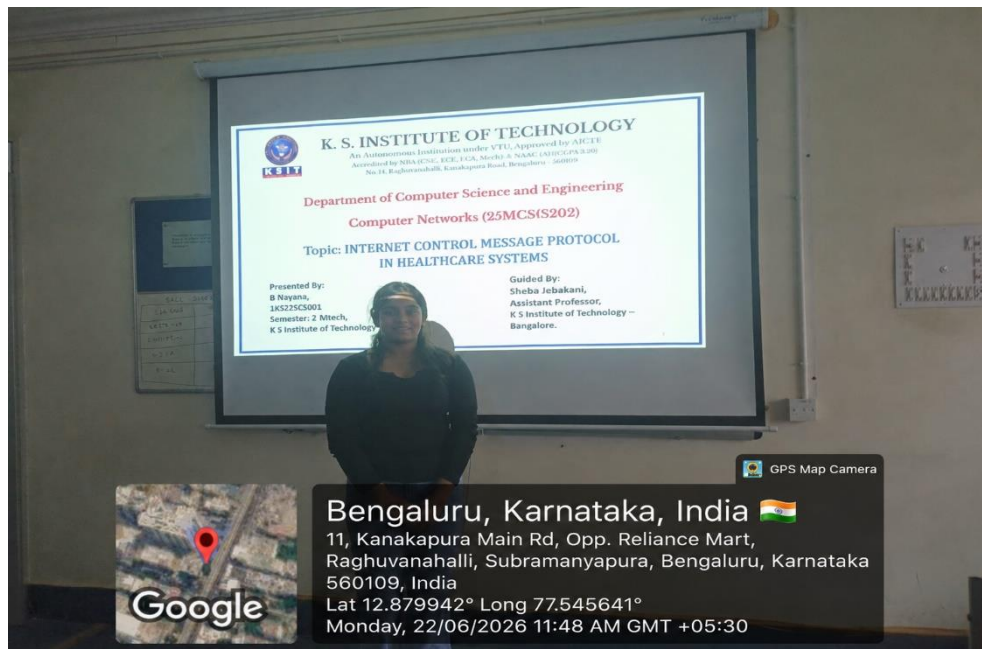


Issued on: Tuesday, June 23, 2026
To verify, scan the QR code at <https://verify.onwingspan.com>



Congratulations! You make us proud!

Satheesha B.N.
Satheesha B. Nanjappa
Senior Vice President and Head
Education, Training and Assessment
Infosys Limited



REFERENCES

- [1] [RFC 792: Internet Control Message Protocol | RFC Editor](#)
- [2] [Wireshark User's Guide](#)
- [3] [ping | Microsoft Learn](#)
- [4]. [Free Computer Networking Course by Cisco: Start Learning Now](#)
- [5]. <https://www.geeksforgeeks.org/internet-control-message-protocol-introduction/>